



AngularJS is an open source Web application framework maintained by Google and the community, which helps to build Single Page Applications (SPA). Let's get to know it better.

AngularJS can be introduced as “a front-end framework capable of incorporating the dynamicity of JavaScript with HTML.” The self-proclaimed ‘super heroic’ JavaScript MVW (Model View Whatever) framework is maintained by Google and many other developers at Github. This open source framework works its magic on Web applications of the Single Page Applications (SPA) category. The logic behind an SPA is that an initial page is loaded at the start of an application from the server. When an action is performed, the application fetches the required resources from the server and adds them to the initial page. The key point here is that an SPA just makes one server round trip, providing you with the initial page. This makes your applications very responsive.

Why AngularJS?

AngularJS brings out the beauty in Web development. It is extremely simple to understand and code. If you're familiar with HTML and JavaScript, you can write

the ‘Hello World’ program in minutes. With the help of Angular, the combined power of HTML and JavaScript can be put to maximum use. One of the prominent features of Angular is that it is extremely easy to test. And that makes it very suitable for creating large-scale applications. Also, the Angular community, comprising Google's developers primarily, is very active in the development process. Google Trends gives assuring proof of Angular's future in the field of Web development (Figure 1).

Core features

Before getting into the basics of AngularJS, you need to understand two key terms—*templates* and *models*. The HTML page that is rendered out to you is pretty much the template. So basically, your template has HTML, Angular entities (directives, filters, model variables, etc) and CSS (if necessary). The example code given below for data binding is a template.

In an SPA, the data and presentation of data is separated by a model layer that handles data and a view layer that reads